

sparse_data_structures

October 25, 2024

1 Sparse Matrix data structures

We consider the following simple matrix.

```
[1]: import numpy as np

A = np.array([
    [1, 0, 0, 2, 0],
    [3, 4, 0, 5, 0],
    [6, 0, 7, 8, 9],
    [0, 0, 10, 11, 0],
    [0, 0, 0, 0, 12]
])
print(A)
```

```
[[ 1  0  0  2  0]
 [ 3  4  0  5  0]
 [ 6  0  7  8  9]
 [ 0  0 10 11  0]
 [ 0  0  0  0 12]]
```

1.1 The COO (Coordinate) format

We start with the COO format. It is the most simple format. Let us convert the matrix into it.

```
[2]: from scipy.sparse import coo_matrix

A_coo = coo_matrix(A)
```

- The coo format is a very simple format that explicitly stores the row entries.
- It consists of three arrays
 - the row indices
 - the column indices
 - the data entries.

```
[3]: print(A_coo.row)
print(A_coo.col)
print(A_coo.data)
```

```
[0 0 1 1 1 2 2 2 2 3 3 4]
[0 3 0 1 3 0 2 3 4 2 3 4]
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

We can easily print out the triplets of row index, column index and associated data entry.

```
[4]: list(zip(A_coo.row, A_coo.col, A_coo.data))
```

```
[4]: [(0, 0, 1),
      (0, 3, 2),
      (1, 0, 3),
      (1, 1, 4),
      (1, 3, 5),
      (2, 0, 6),
      (2, 2, 7),
      (2, 3, 8),
      (2, 4, 9),
      (3, 2, 10),
      (3, 3, 11),
      (4, 4, 12)]
```

- Simplest format and most obvious format.
- The coo format in Scipy is most frequently used for the generation of sparse matrices.
- However, coo is not a suitable format for typical matrix operations. Also, it is not yet optimal in terms of storage requirements.
- Row indices repeated for same column.

1.2 The CSR (Compressed Sparse Row) Format

- Store an array of index pointers that give the starting position of the row within the column array.

Let us demonstrate how this works.

We first convert the COO matrix format into the CSR format.

```
[5]: A_csr = A_coo.tocsr()
```

Let us now print out the arrays that define the CSR format. We have three arrays.

- `A_csr.data` - The data array containing the nonzero entries
- `A_csr.indices` - The column indices for the nonzero entries
- `A_csr.indptr` - Pointers into the column indices to store which indices belong to which row.

The first two are the same as in the COO format.

The last one requires explanation.

For this let us print out the three arrays.

```
[6]: print(A_csr.data)
      print(A_csr.indices)
      print(A_csr.indptr)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
[0 3 0 1 3 0 2 3 4 2 3 4]
[ 0  2  5  9 11 12]
```

Comparing the arrays shows that the first two are indeed identical to the corresponding arrays for the COO format.

The `indices` array give the location of columns and data for a particular row like so:

```
# Columns
indices[indptr[i] : indptr[i + 1]]
# Data
data[indptr[i] : indptr[i + 1]]
```

```
[7]: row = 2
      indices = A_csr.indices
      indptr = A_csr.indptr
      data = A_csr.data
      print(indices[indptr[row]:indptr[row+1]])
      print(data[indptr[row]:indptr[row+1]])
```

```
[0 2 3 4]
[6 7 8 9]
```

```
[8]: coo_rows = A_coo.row == 2
      coo_cols = A_coo.col[coo_rows]
      coo_data = A_coo.data[coo_rows]
      print(coo_cols)
      print(coo_data)
```

```
[0 2 3 4]
[6 7 8 9]
```

Variant that works on columns

- This is called CSC (Compressed Sparse Column) Format.

Both CSC and CSR are widely used in software for large sparse matrices.

1.3 CSR Matrix-vector products

One of the benefits of the CSR format is the simple implementation for the matrix-vector product.

- It naturally parallelises on multithreaded CPUs.

```
[9]: import numba

@numba.jit(nopython=True, parallel=True, fastmath=True)
def csr_matvec(data, indices, indptr, shape, x):
```

```

"""Evaluates the matrix-vector product with a CSR matrix."""
# Get the rows and columns

m, n = shape

y = np.zeros(m, dtype=np.float64)

for row_index in numba.prange(m):
    col_start = indptr[row_index]
    col_end = indptr[row_index + 1]
    v = 0.0
    for col_index in range(col_start, col_end):
        v += data[col_index] * x[indices[col_index]]
    y[row_index] = v

return y

```

Let's test this against the Scipy provided. We use the matrix generated with the `discretise_poisson` routine.

```

[10]: from scipy.sparse import coo_matrix

def discretise_poisson(N):
    """Generate the matrix and rhs associated with the discrete Poisson_
    operator."""

    nelements = 5 * N**2 - 16 * N + 16

    row_ind = np.empty(nelements, dtype=np.float64)
    col_ind = np.empty(nelements, dtype=np.float64)
    data = np.empty(nelements, dtype=np.float64)

    f = np.empty(N * N, dtype=np.float64)

    count = 0
    for j in range(N):
        for i in range(N):
            if i == 0 or i == N - 1 or j == 0 or j == N - 1:
                row_ind[count] = col_ind[count] = j * N + i
                data[count] = 1
                f[j * N + i] = 0
                count += 1
            else:
                row_ind[count : count + 5] = j * N + i
                col_ind[count] = j * N + i

```

```

        col_ind[count + 1] = j * N + i + 1
        col_ind[count + 2] = j * N + i - 1
        col_ind[count + 3] = (j + 1) * N + i
        col_ind[count + 4] = (j - 1) * N + i

        data[count] = 4 * (N - 1)**2
        data[count + 1 : count + 5] = - (N - 1)**2
        f[j * N + i] = 1

        count += 5

    return coo_matrix((data, (row_ind, col_ind)), shape=(N**2, N**2)).tocsr(), f

```

```

[11]: import numpy as np
N = 1_000

A, _ = discretise_poisson(N)

print(f'Shape of matrix {A.shape}')

# Generate a random vector
rand = np.random.RandomState(0)
x = rand.randn(N * N)

y = csr_matvec(A.data, A.indices, A.indptr, A.shape, x)

# Compare with the Scipy sparse matrix multiplication

y_exact = A @ x
rel_error = np.linalg.norm(y - y_exact, np.inf) / np.linalg.norm(y_exact, np.
    ↪inf)
print(f"Error: {round(rel_error, 2)}.")

```

Shape of matrix (1000000, 1000000)

Error: 0.0.

Let us time our implementation against the Scipy one.

```

[12]: # Our implementation
%timeit y = csr_matvec(A.data, A.indices, A.indptr, A.shape, x)

```

4.79 ms ± 355 s per loop (mean ± std. dev. of 7 runs, 100 loops each)

```

[13]: # The default Scipy implementation
%timeit y = A @ x

```

8.33 ms ± 656 s per loop (mean ± std. dev. of 7 runs, 100 loops each)

We can see a small improvement against the default Scipy implementation.

1.4 Other sparse formats.

There are a number of sparse matrix formats and Scipy is supporting several of them. More information on sparse matrix classes and operations for handling sparse matrices can be found at <https://docs.scipy.org/doc/scipy/reference/sparse.html>.